



NyayaSetu (न्यायसेतु) — Project Report

AI-Assisted Smart Court Case Scheduling & Cause-List Optimisation Platform

Field	Detail
Project Name	NyayaSetu (न्यायसेतु)
Event	Smart India Hackathon (SIH) 2026
Target Users	Indian District & Taluka Courts
Built By	Team NyayaSetu — Sujal Javeri
Version	1.0.0
Date	September 2026

Table of Contents

- [1. What is NyayaSetu?](#)
- [2. The Problem We Are Solving](#)
- [3. Our Solution — In Simple Words](#)
- [4. Who Uses NyayaSetu?](#)
- [5. Complete Feature List](#)
- [6. Technology Stack — Every Single Tool We Used](#)
- [7. How the System Works — Step by Step](#)
- [8. Database Design](#)
- [9. Project File Structure](#)
- [10. What Makes NyayaSetu Different From Others](#)
- [11. Security & Compliance](#)
- [12. How to Run the Project](#)
- [13. Conclusion](#)

1. What is NyayaSetu?

NyayaSetu (meaning "Bridge to Justice" in Hindi) is a **web-based platform** that helps Indian courts manage their daily work better. It uses **smart algorithms and AI assistance** to schedule court

hearings, prevent mistakes like double-booking judges or courtrooms, and make sure urgent cases are heard first.

Think of it like a **smart assistant for court registrars** — the people who manage the daily schedule of a court. Instead of manually assigning cases to judges and courtrooms (which leads to mistakes and delays), NyayaSetu does the heavy lifting automatically while always keeping humans in control of the final decision.

2. The Problem We Are Solving

India's Court Backlog Crisis

India's judicial system currently has **over 50 million pending court cases**. This is one of the biggest challenges facing the country today. A huge part of this problem comes from how courts manage their daily work:

Problem	What Happens
Manual Scheduling	Court staff manually assign cases to judges and courtrooms using paper registers and spreadsheets. This is slow and full of human error.
Double Booking	A judge gets accidentally assigned to two hearings at the same time. One hearing gets cancelled, wasting everyone's time.
Courtroom Clashes	Two different cases are assigned to the same courtroom at the same time.
Unfair Prioritisation	Urgent cases (like child protection or senior citizen matters) don't get priority over routine cases.
No Emergency Planning	If a judge falls sick, there's no quick way to reassign all their hearings. Staff have to do it manually.
No Transparency	Litigants (the people involved in cases) have no easy way to check when their next hearing is.
No Audit Trail	There's no record of why a particular case was scheduled at a particular time, making it hard to ensure accountability.

Why Existing Solutions Fail

Most existing court management software simply digitises the register — it moves the paper to a computer screen but doesn't actually make the scheduling smarter. They also rely on "black box" AI

that gives recommendations without explaining why, which courts cannot trust for legal decisions.

3. Our Solution — In Simple Words

NyayaSetu solves these problems with a **three-layer approach**:

Layer 1: Smart Scheduling Engine (The Brain)

A **deterministic, rules-based algorithm** that checks every possible combination of Judge + Courtroom + Time Slot and finds the best one. It works like this:

1. **First, it eliminates invalid options** — If a judge is on leave, or a courtroom is closed, or the case is too long to fit the time slot, that option is thrown out. These are called **Hard Constraints**.
2. **Then, it ranks the remaining options** — Among all valid options, it scores each one based on how well the judge's expertise matches the case type, how balanced the workload is, and how urgent the case is. These are called **Soft Preferences**.
3. **Finally, it shows the best options to the registrar** — The registrar sees the top recommendations with a full explanation of why each was chosen, and picks the one they want.

Layer 2: AI Copilot (The Assistant)

A natural language chatbot powered by **Google Gemini 3.5 Flash** (with fallback to Groq and OpenAI) that answers questions about the court's caseload, schedules, legal provisions, and more. Think of it as a helpful colleague who knows everything about the court's current state.

Layer 3: Public Access (The Bridge)

A simple, no-login-required portal where any citizen can type in their case number and see their next hearing date, judge, courtroom, and position in the cause list — available in **English and Hindi**.

4. Who Uses NyayaSetu?

NyayaSetu is designed for **four different types of users**, each with their own workspace:

Role	What They Can Do	Login Required?
Administrator	Full control: manage judges, courtrooms, user accounts, priority scoring weights, and system audit logs.	✓ Yes
Registrar	Day-to-day work: register cases, run smart scheduling, resolve conflicts, use What-If simulation, publish cause lists, and notify parties.	✓ Yes
Judge	View their own cause list for any date, see why cases are ordered the way they are, check their hearing calendar, and directly pull cases onto their bench.	✓ Yes
Public / Litigant	Check case status by entering a case number. No account needed. Available in multiple languages.	✗ No

5. Complete Feature List

Feature 1: Multi-Constraint Smart Scheduling Solver

This is the core engine. When a registrar needs to schedule a hearing, the engine evaluates every possible Judge + Courtroom + Time Slot combination and checks **6 Hard Constraints** (pass/fail rules):

#	Hard Constraint	What It Checks
1	Judge Availability	Is the judge marked as available on that date and slot?
2	Courtroom Availability	Is the courtroom open and functional?
3	No Judge Double-Booking	Is the judge already sitting in another hearing at the same time?
4	No Courtroom Double-Booking	Is the courtroom already occupied at the same time?
5	Slot Duration Fit	Can the estimated hearing duration fit within the slot length?
6	Judge Workload Ceiling	Would this push the judge above the maximum active hearings threshold (default: 25)?
7	Court Holiday Check	Is the date a gazetted holiday or non-sitting day?

After passing all hard constraints, the valid options are ranked using **4 Soft Preference factors**:

Factor	Default Weight	What It Measures
Specialisation Fit	35%	Does the judge's expertise match the case category (e.g., POCSO, Commercial, Civil)?
Workload Balance	30%	How evenly distributed is work across judges? Prefers judges with fewer active cases.
Priority Slot Match	20%	Are high-priority cases being given earlier slots?
Courtroom Utilisation	15%	Are courtrooms being used efficiently? Prefers less-busy rooms.

All weights are **admin-configurable** — the court administrator can adjust these percentages from the settings panel.

Feature 2: Explainable Decision Receipts

Every scheduling recommendation comes with a **transparent audit ticket** — a card that shows:

-  All 6 hard constraints that were passed (with green checkmarks)
-  The exact score breakdown for each soft preference factor (e.g., "+28.0 / 35 for Specialisation fit")
-  A deterministic confidence score (0–100%)
-  The total number of combinations evaluated, valid candidates found, and reasons for rejection

Why this matters: Unlike "black box" AI that just says "do this", NyayaSetu tells you exactly *why* it recommends something. A registrar or judge can verify the logic themselves.

Feature 3: Alternative Scheduling Options

The engine doesn't just give one answer — it shows **multiple valid alternatives** (Option 2, Option 3, Option 4...) so the registrar can compare and pick the best fit. Each alternative has its own score and explanation. The registrar can select any option with a **single click**.

Feature 4: Custom Judicial Directive & Judge Self-Scheduling

Sometimes, a judge or registrar needs to override the system and schedule a case manually. NyayaSetu supports this with:

- **Custom Judicial Schedule Modal:** A form where you pick a specific Judge, Courtroom, and Slot for a case. It includes pre-built directive presets like:
 - "Urgent Mention allowed by Bench"
 - "Part-heard matter fixed per Bench order"
 - "Special Sitting requested by Judge"
- **Live Pre-Flight Conflict Check:** Before you confirm, the system runs all hard constraints in real-time and shows any conflicts. If there's a clash, you'll know immediately.
- **Judicial Override Justification:** If you proceed despite a conflict (override), the system logs the reason to the immutable audit trail.

Judge's Bench Portal (`/bench`): Judges have a dedicated page where they can:

- View their daily cause list
 - See the reasoning behind the listing order
 - Search all open cases in the registry and directly pull a case onto their bench
-

Feature 5: Zero-Risk What-If Simulation (Digital Twin)

This is one of the most powerful features. It answers the question: "**What happens if a judge suddenly becomes unavailable?**" or "**What if a courtroom has to close for maintenance?**"

How it works:

1. The registrar selects a judge and a date (or a courtroom and a date)
2. The system creates a **deep copy** of the entire schedule in memory (a "digital twin")
3. It finds every hearing affected by the disruption
4. For each affected hearing, it runs the scheduling engine to find alternative slots
5. The registrar can review all proposed reassignments and **commit them with one click**
6. Only when the registrar confirms does the system write changes to the actual database

Zero risk: Because everything runs on a copy first, the live schedule is never touched during exploration.

Feature 6: 8-Factor Statutory Priority Scoring Engine

Every case gets a priority score from **0 to 100**, calculated using 8 factors aligned with Government of India mandates:

#	Factor	Max Weight	What It Considers
1	Case Category Urgency	30 pts	Different case types have different urgency levels (e.g., Criminal > Civil)
2	Pending Duration	40 pts	How long the case has been pending (capped at 365 days for scoring)
3	Previous Adjournments	20 pts	How many times the case has been postponed (capped at 5)
4	FTSC / POCSO Act	30 pts	Is this a Fast Track Special Court or child protection case?
5	Senior Citizen Litigant	12 pts	Does the case involve a senior citizen?
6	Property Dispute (5+ years)	10 pts	Is this a long-pending property case?
7	Statutory Limitation Deadline	18 pts	Is the legal deadline approaching?
8	Administrative Priority Boost	10 pts	Has an administrator flagged this case as urgent?

Based on the score, cases are classified into:

- **Tier 1 (High Urgency):** Score ≥ 70
- **Tier 2 (Moderate):** Score 40–69
- **Tier 3 (Normal):** Score < 40

All weights are **fully configurable** by the court administrator.

Feature 7: 3-Stage Procedural Cause List Optimiser

The system generates balanced daily cause lists organised into three judicial sitting stages, following the standard Indian court schedule:

Stage	Time	Types of Matters
Stage 1 (Morning)	10:30 AM – 11:30 AM	Urgent Mentions, Bail Applications, Fresh Admissions
Stage 2 (Midday)	11:30 AM – 01:30 PM	Contested Arguments, Framing of Issues, Evidence
Stage 3 (Afternoon)	02:30 PM – 04:30 PM	Final Orders, Pronouncements, Miscellaneous Disposals

Features include:

- **Drag-and-drop reordering:** Registrars can manually adjust the order
- **"Why This Order?"** button: Shows exactly why each case is listed in its position
- **PDF export:** Generate branded NyayaSetu PDF cause lists
- **Filter by judge, courtroom, or stage**

Feature 8: Full-Page Hindi Translation (EN ⇌ हि)

NyayaSetu supports **complete bilingual operation** using a dual-engine approach:

1. **Instant DOM TreeWalker Engine:** A dictionary of **500+ Indian legal, procedural, and UI terms** that translates text in real time by walking through every text node in the page. This covers navigation, buttons, labels, status badges, legal terminology, and more.
2. **Dynamic Translation Layer:** For longer content like case titles, party names, and AI responses, a seamless integration translates content that the dictionary doesn't cover.
3. **Lossless Toggle:** A single click on the **EN ⇌ हि** button in the top bar switches the entire application between English and Hindi instantly. The original text is preserved so you can switch back without any data loss.

Feature 9: Real-Time Impact Counter Banner

The Dashboard shows **animated live counters** that display the system's impact:

- Conflicts Detected & Prevented
- Tier 1 Cases Prioritised

- AI Recommendations Issued
- Active Scheduled Hearings

These numbers come from the actual database — they're not hardcoded or estimated.

Feature 10: AI Registry Copilot (NLP Q&A)

A chatbot built into the sidebar that can answer natural language questions about the court's operations. Examples:

- "How many cases are pending?"
- "What is Judge Sharma's current workload?"
- "Are there any scheduling conflicts today?"
- "What is Section 138 of NI Act?"

Architecture:

- **Primary:** Google Gemini 3.5 Flash
- **Fallback 1:** Groq Cloud API (multiple models including GPT-OSS-120B, Compound-Mini)
- **Fallback 2:** Custom / Local LLM (supports Ollama, LM Studio, vLLM)
- **Fallback 3:** OpenAI GPT-4o Mini
- **Baseline:** Deterministic rule-based handler (works even when all AI providers are down)

The AI is given a **live snapshot of the database** (cases, judges, courtrooms, schedules, conflicts) in its system prompt, so its answers are always grounded in real data.

Security: User input is sanitised, rate-limited (30 queries/minute), and wrapped in `<user_query>` tags to prevent prompt injection.

Feature 11: Conflict Detection Scanner

A real-time scanner that checks every active schedule in the system and flags any conflicts:

Conflict Type	Description
Judge Double-Booked	Same judge assigned to two overlapping hearings
Courtroom Double-Booked	Same courtroom hosting two overlapping hearings
Judge Unavailable	Judge marked unavailable but still has a scheduled hearing
Courtroom Unavailable	Courtroom marked unavailable but still has a scheduled hearing

Slot Already Allocated	Two different cases assigned to the same slot
Duration Overflow	Case hearing duration exceeds the slot length
Workload Exceeded	Judge has more active hearings than the configured maximum
Holiday Closure	Hearing scheduled on a gazetted holiday or non-sitting day

Feature 12: Backlog Simulator (FIFO vs Priority Comparison)

A projection tool that answers: **"What if we continue scheduling cases in filing-date order vs. our priority-based order?"**

It runs a deterministic in-memory simulation over 6-month and 12-month horizons and shows:

- How fast the backlog reduces under each approach
 - Average wait time for Tier 1 cases
 - Number of statutory deadline breaches
 - Side-by-side Recharts comparison graphs
-

Feature 13: Governance & Compliance Dashboard

Tracks Supreme Court AI Regulation alignment:

- **Recommendation Acceptance Rate:** What percentage of AI recommendations were accepted, modified, or rejected
 - **Human Override Log:** Every time a registrar overrides the system, with justification
 - **Priority Tier Distribution:** Breakdown of cases by tier
 - **Tier 1 Time-to-Hearing:** Average time from case registration to first hearing for high-priority cases
 - **Full Audit Trail:** Every action timestamped with user identity
-

Feature 14: Branded PDF Exports

Professional PDF documents generated using jsPDF and jsPDF-AutoTable:

- Cause lists with NyayaSetu branding
- Hearing schedules

- Case reports
- Priority score breakdowns

Every PDF includes a header with the NyayaSetu brand, generation timestamp, and "Internal registry document — decision support only" footer.

Feature 15: Public Case Status Lookup

A **no-login portal** at `/case-status` where anyone can:

- Enter their case number
 - See current status, next hearing date, assigned judge, courtroom
 - View their position in the day's cause list
 - Available in **English and Hindi**
-

6. Technology Stack — Every Single Tool We Used

Frontend & Fullstack SSR

Technology	Version	Why We Used It
React	19.2	The UI library for building interactive interfaces. React 19 brings improved server-side rendering and concurrent features.
TanStack Start	1.168	A fullstack React framework with server-side rendering (SSR), server functions, and isomorphic code execution.
TanStack Router	1.170	File-based routing system. Each page is a file in <code>src/routes/</code> . Provides type-safe navigation.
TanStack Query	5.101	Data fetching and caching library. Automatically refetches data when it becomes stale, handles loading/error states.
Vite	8.2	Lightning-fast build tool and dev server. Provides instant Hot Module Replacement (HMR) during development.
Nitro Engine	3.0 (beta)	The server engine behind TanStack Start. Handles SSR, API routes, and deployment targets.

Language & Type Safety

Technology	Version	Why We Used It
TypeScript	5.8 (Strict Mode)	Type-safe JavaScript. Catches bugs at compile time. We run in strict mode with zero type errors .
Zod	3.24	Runtime schema validation. Used to validate user inputs, API payloads, and server function parameters.

Styling & UI

Technology	Version	Why We Used It
Tailwind CSS	v4.2	Utility-first CSS framework. Provides consistent, responsive design without writing custom CSS classes.
Radix UI	Multiple packages	Accessible UI primitives (modals, dropdowns, tabs, tooltips, etc.). Handles keyboard navigation, focus management, and ARIA attributes automatically.
Lucide React	0.575	Beautiful, consistent SVG icon library. 1500+ icons.
Sonner	2.0	Toast notification library for showing success/error messages.
tw-animate-css	1.3	CSS animations integrated with Tailwind.
class-variance-authority (CVA)	0.7	Component variant management. Lets us define button styles like <code>variant="destructive"</code> or <code>size="sm"</code> .
tailwind-merge	3.5	Intelligently merges Tailwind CSS classes to prevent conflicts.

Data Visualisation

Technology	Version	Why We Used It
Recharts	2.15	React charting library for bar charts, line charts, and area charts. Used in the Dashboard, Backlog Simulator, and Governance pages.

Database & Authentication

Technology	Why We Used It
------------	----------------

Supabase	Backend-as-a-service providing PostgreSQL database, authentication, and real-time subscriptions.
PostgreSQL	Enterprise-grade relational database. Stores all cases, judges, courtrooms, schedules, audit logs, and AI recommendations.
Row Level Security (RLS)	Supabase's PostgreSQL policies that enforce access control at the database level. For example, only admins can modify judge records.
Database Triggers	Automatically run functions when data changes. For example, auto-creating a user profile when someone signs up.

AI Layer

Technology	Why We Used It
Google Gemini 3.5 Flash	Primary AI model for the Registry Copilot. Fast, accurate, and cost-effective.
Groq Cloud API	Ultra-fast backup AI provider. Supports models like GPT-OSS-120B and Compound-Mini.
OpenAI GPT-4o Mini	Final fallback AI provider.
Custom/Local LLM Support	Can connect to self-hosted models via Ollama, LM Studio, or vLLM for courts that need data sovereignty.

PDF Generation

Technology	Version	Why We Used It
jsPDF	4.2	Client-side PDF generation in JavaScript. Creates A4-format PDFs with headers, footers, and tables.
jsPDF-AutoTable	5.0	Plugin for jsPDF that creates professional formatted tables in PDFs.

Forms & Validation

Technology	Version	Why We Used It
React Hook Form	7.71	Performant form management with minimal re-renders.

@hookform/resolvers	5.2	Connects Zod schemas to React Hook Form for automatic validation.
---------------------	-----	---

Date & Time

Technology	Version	Why We Used It
date-fns	4.1	Modern JavaScript date utility library. Formatting, parsing, and comparing dates.
React Day Picker	9.14	Date picker calendar component.

Other Libraries

Technology	Version	Why We Used It
react-markdown	10.1	Renders AI Copilot responses as formatted Markdown (bold, lists, headings).
remark-gfm	4.0	GitHub Flavored Markdown support for tables and task lists in AI responses.
react-resizable-panels	4.6	Draggable panel layouts for the scheduling workbench.
embla-carousel-react	8.6	Carousel component for card-based UI.
input-otp	1.4	One-time password input component (for authentication).
cmdk	1.1	Command palette (⌘K) component for quick search.
vaul	1.1	Mobile-friendly drawer component.
lru-cache	11.5	In-memory cache with Least Recently Used eviction. Used for rate limiting.

Development Tools

Technology	Version	Why We Used It
ESLint	9.32	Code quality linter. Catches bugs and enforces coding standards.

Prettier	3.7	Code formatter. Ensures consistent code style across the team.
@vitejs/plugin-react	5.2	Vite plugin for React support (Fast Refresh, JSX transform).

7. How the System Works — Step by Step

Step 1: Case Registration

A registrar enters a new case with details like case number, parties, category (Criminal, Civil, POCSO, etc.), filing date, and flags (senior citizen, FTSC, property dispute, statutory limitation deadline).

Step 2: Priority Scoring

The system immediately calculates a **0–100 priority score** using the 8-factor engine and assigns a tier (Tier 1, 2, or 3).

Step 3: Smart Scheduling

When the registrar wants to schedule a hearing:

1. The engine loads all judges, courtrooms, time slots, availability records, and existing schedules from the database.
2. It evaluates every possible (Judge, Courtroom, Slot) combination.
3. Invalid combinations are filtered out by hard constraints.
4. Valid combinations are scored and ranked by soft preferences.
5. The top recommendations are presented with full explanations.

Step 4: Registrar Decision

The registrar reviews the recommendations, checks the decision receipt, and either:

- **Accepts** the top recommendation
- **Selects an alternative** option
- **Manually overrides** using the Custom Judicial Schedule Modal

Step 5: Conflict Check

The system continuously scans for conflicts. If a new conflict is detected (e.g., due to a judge being marked unavailable after scheduling), it flags it immediately.

Step 6: Cause List Generation

The system generates daily cause lists ordered by:

- 1. Manual overrides (if any registrar has manually set positions)
- 2. Priority tier (Tier 1 first)
- 3. Priority score (highest first within the same tier)
- 4. Slot time (earlier first)

Step 7: What-If Simulation (If Needed)

If an emergency occurs (e.g., judge falls sick), the registrar can run a simulation to see which hearings are affected and get automatic reassignment suggestions.

Step 8: Audit & Governance

Every action is logged with the user's identity and timestamp. The governance dashboard tracks AI recommendation acceptance rates and human override patterns.

8. Database Design

NyayaSetu uses **Supabase PostgreSQL** with the following core tables:

Table	Purpose	Key Columns
profiles	User profiles	id , full_name , permissions
user_roles	Role assignments	user_id , role (admin / registrar)
judges	Judge registry	name , specialisation , current_workload
courtrooms	Courtroom registry	name , capacity , type , current_allocation
case_categories	Case type definitions	name , typical_duration_minutes , urgency_weight
cases	All registered cases	case_number , parties , status , filing_date , priority_score , priority_tier , is_ftsc_pocso , senior_citizen_litigant ,

		property_dispute_5yr_plus , statutory_limitation_deadline
hearing_slots	Available time slots	date , start_time , end_time
schedules	Hearing assignments	case_id , judge_id , courtroom_id , slot_id , status , cause_list_position
availability	Judge/courtroom availability	entity_type , entity_id , date , slot_id , status
ai_recommendations	AI scheduling recommendations	schedule_id , reasoning , status (accepted/modified/rejected)
audit_logs	Full activity trail	user_id , action , entity_affected , timestamp
priority_settings	Configurable scoring weights	All 8 priority weights + scheduling weights + workload caps
court_holidays	Gazetted holidays	date , name , type , jurisdiction

Security:

- **Row Level Security (RLS)** is enabled on every table
- Only admins can modify judges, courtrooms, and user roles
- All authenticated users can view data
- Service role has full access for server-side operations

Enums:

- `app_role` : admin, registrar
- `case_status` : filed, scheduled, in_progress, adjourned, disposed
- `entity_type` : judge, courtroom
- `availability_status` : available, unavailable
- `schedule_status` : proposed, confirmed, completed, cancelled
- `recommendation_status` : accepted, modified, rejected

9. Project File Structure

— index.tsx	# Landing / portal page (role selection)
— auth.tsx	# Login page (Supabase authentication)
— case-status.tsx	# Public case status lookup (no login)
— setup-admin.tsx	# First-time admin setup
— _authenticated/	# Protected routes (login required)
— dashboard.tsx	# Main dashboard with metrics and charts
— smart-scheduling.tsx	# Scheduling engine workbench
— conflicts.tsx	# Real-time conflict scanner
— what-if-simulation.tsx	# Digital twin simulation sandbox
— cause-list.tsx	# 3-Stage cause list generator
— bench.tsx	# Judge's personal portal
— calendar.tsx	# Hearing calendar with PDF export
— backlog-simulator.tsx	# FIFO vs Priority comparison tool
— governance.tsx	# AI compliance and audit metrics
— reports.tsx	# Case reports and analytics
— admin.tsx	# Administration panel
— priority-settings.tsx	# Configurable priority weights UI
— activity-log.tsx	# Full audit trail viewer
— cases/	# Case management pages
— judges/	# Judge management pages
— courtrooms/	# Courtroom management pages
— lib/	# Core algorithms, data queries & server functions
— scheduling.ts	# ★ Multi-constraint scheduling engine (521 lines)
— conflicts.ts	# ★ Conflict detection engine (393 lines)
— priority.ts	# ★ 8-factor priority

```
scoring engine (315 lines)
|   |   |— simulation.ts           # ★ What-If digital twin
simulation (350 lines)
|   |   |— cause-list.ts         # Cause list ordering and
persistence
|   |   |— i18n.tsx              # Dual-engine Hindi/English
translation (435 lines)
|   |   |— ai.server.ts          # Multi-provider AI query
layer (Gemini/Groq/OpenAI)
|   |   |— assistant.ts         # Deterministic Q&A rule
handler
|   |   |— assistant.functions.ts # AI Copilot server function
with live DB snapshot
|   |   |— governance.ts        # Governance metrics
computation
|   |   |— pdf.ts               # Branded PDF document
generator
|   |   |— audit.ts             # Immutable audit trail
system
|   |   |— recommendations.ts   # Recommendation persistence
and reasoning
|   |   |— backlog-simulation.ts # FIFO vs Priority backlog
projector
|   |   |— availability.ts       # Availability management
queries
|   |   |— batch-scheduling.ts   # Batch scheduling operations
|   |   |— briefing.ts          # Daily registry briefing
generator
|   |   |— calendar.ts          # Calendar data queries
|   |   |— cases.ts             # Case data management
|   |   |— dashboard.ts         # Dashboard metrics queries
|   |   |— holidays.ts          # Gazetted court holidays
|   |   |— notifications.ts     # In-app notification system
|   |   |— predictions.ts       # Case outcome predictions
|   |   |— registry.ts          # Judge/courtroom registry
|   |   |— reports.ts           # Report generation queries
|   |   |— utilisation.ts        # Resource utilisation
metrics
|   |   |— why-this-order.ts     # Cause list order explainer
|   |   |— rate-limit.server.ts # Rate limiting for AI
```

```

queries
|   |   └─ security.server.ts           # Input sanitisation &
injection defence
|   |   └─ error-capture.ts             # Error handling utilities
|   |
|   └─ integrations/                    # External service
connections
|   |   └─ supabase/                    # Supabase client and types
|   |
|   └─ styles.css                        # Tailwind v4 theme tokens
and custom animations
|   └─ router.tsx                       # TanStack Router
configuration
|   └─ server.ts                         # Server entry point
|   └─ start.ts                         # Application bootstrap
|
└─ supabase/                            # Database
   └─ supabase_setup.sql                 # Complete schema, RLS
policies, triggers (1000+ lines)
|   └─ migrations/                       # Database migration files
|
└─ docs/                                # Documentation
   └─ nyayasetu_sih_report.md            # SIH submission report
   └─ SIH_APP_AUDIT_REPORT.md            # Application audit report
|
└─ public/                              # Static assets
└─ scripts/                             # Build scripts
└─ package.json                         # Dependencies and scripts
└─ vite.config.ts                       # Vite build configuration
└─ tsconfig.json                        # TypeScript configuration
└─ AGENTS.md                            # Developer guidelines
└─ README.md                            # Project documentation

```

10. What Makes NyayaSetu Different From Others

This is the most important section. Here's what sets NyayaSetu apart from every other court management system:

1. No Black-Box AI — Fully Deterministic & Explainable

Others: Most AI-powered scheduling tools use machine learning models that give a recommendation but can't explain *why*. In a judicial context, this is unacceptable — judges and registrars need to trust and verify every decision.

NyayaSetu: Our scheduling engine is **100% deterministic**. There is **no AI, no model call, and no randomness** in the scheduling algorithm. The same inputs always produce the same ranked output. Every recommendation comes with a **Decision Receipt** that breaks down exactly which constraints were checked and how scores were calculated. AI is only used for the optional conversational copilot — never for the actual scheduling decisions.

From our code: "IMPORTANT: there is NO AI, no model call and no randomness anywhere in this file. Every recommendation is produced by pure functions over the registry data."

2. Human-in-the-Loop — Always

Others: Many systems auto-schedule cases without human approval, or make it hard to override.

NyayaSetu: Every recommendation requires **explicit human approval**. The registrar can accept, select an alternative, or completely override with a manual assignment. Every override is logged with a justification. The system is a **decision support tool**, not a decision maker.

3. Zero-Risk Digital Twin Simulation

Others: If a judge falls sick, courts scramble to manually reassign all affected hearings. No existing solution offers a way to preview the impact before making changes.

NyayaSetu: Our What-If Simulation creates an **in-memory clone** of the entire schedule, runs the disruption scenario, finds alternatives for every affected hearing, and lets the registrar review and commit — all without touching the live schedule until they're ready.

4. 8-Factor Statutory Priority Scoring (India-Specific)

Others: Most systems use simple priority flags (High/Medium/Low) or date-based ordering.

NyayaSetu: We compute a **granular 0–100 score** based on 8 factors specifically aligned with Indian statutory requirements — including POCSO/FTSC mandates, Senior Citizen Act, Limitation

Act deadlines, and long-pending property disputes. Every factor is transparent and admin-configurable.

5. True Bilingual Operation (Not Just Translation)

Others: Some systems offer a translated version, but it's usually a separate build or requires page reload.

NyayaSetu: Our **dual-engine translation system** translates the entire application in real-time with a single click — 500+ legal and judicial terms are translated instantly via DOM TreeWalker, and dynamic content is handled by a seamless integration layer. No page reload, no data loss, instant toggle.

6. Judge Self-Scheduling & Bench Portal

Others: Judges typically have no digital interface — they receive paper cause lists.

NyayaSetu: Judges have a dedicated `/bench` portal where they can view their cause list, understand the reasoning behind case ordering, see their hearing calendar, and directly pull cases from the registry onto their bench — all while being restricted to their own scope.

7. Supreme Court AI Regulation Compliance

Others: No existing court management tool in India tracks compliance with the Supreme Court's Draft Regulations for AI in Courts.

NyayaSetu: Our Governance Dashboard specifically tracks metrics required by **Regulation 9** — recommendation acceptance rates, human override patterns with justifications, and full audit trails. This makes NyayaSetu **audit-ready from day one**.

8. Multi-Tier AI Fallback with Deterministic Baseline

Others: If the AI service goes down, the system breaks.

NyayaSetu: Our AI Copilot has a **4-tier fallback chain** (Gemini → Groq → Custom LLM → OpenAI). And even if ALL AI providers fail, the system has a **deterministic rule-based handler** that can

answer common questions using database queries alone. The scheduling engine works entirely without AI — it never depends on external services.

9. Comprehensive Security Stack

Others: Basic authentication only.

NyayaSetu: We implement:

- **Row Level Security** on every database table
 - **Role-based access control** (Admin, Registrar, Judge)
 - **Input sanitisation** and prompt injection defence for AI queries
 - **Rate limiting** (30 queries/minute for AI assistant)
 - **Immutable audit logging** (no one can delete or modify audit entries)
-

10. Multi-Persona Access in One Platform

Others: Different systems for different users.

NyayaSetu: A single platform serves 4 distinct user types:

- Administrators get full system control
 - Registrars get the scheduling workbench
 - Judges get a scoped bench portal
 - Public/litigants get a no-login case status lookup
-

11. Security & Compliance

Security Measure	Implementation
Authentication	Supabase Auth with email/password. Court-issued accounts only (no self-registration for judge/registrar).
Authorisation	Role-based: admin, registrar, judge. Enforced at database level via RLS policies.
Row Level Security	Every table has RLS enabled. Policies check <code>has_role()</code> function before allowing data access.

Input Sanitisation	All user inputs sanitised before processing. AI queries wrapped in <code><user_query></code> tags for injection defence.
Rate Limiting	LRU-cache based rate limiter. AI Copilot limited to 30 queries/minute per session.
Audit Trail	Immutable <code>audit_logs</code> table. Every action recorded with user ID, action description, and timestamp.
AI Governance	Recommendation acceptance/rejection tracked. Human override justifications logged.
Error Handling	Centralised error capture. Audit writes never block user actions — failures are logged to console.

12. How to Run the Project

Prerequisites

- **Node.js** v20.x or higher
- **npm** (comes with Node.js)

Installation

```
# 1. Clone the repository
git clone https://github.com/SujalJaveri/NyaySetu.git
cd NyaySetu

# 2. Install dependencies
npm install

# 3. Copy environment variables template
cp .env.example .env
```

Configure Environment Variables

Edit the `.env` file with your credentials:

```
# Supabase (Required)
VITE_SUPABASE_URL="https://your-project.supabase.co"
VITE_SUPABASE_PUBLISHABLE_KEY="your-anon-key"
SUPABASE_URL="https://your-project.supabase.co"
SUPABASE_PUBLISHABLE_KEY="your-anon-key"
SUPABASE_SERVICE_ROLE_KEY="your-service-role-key"

# AI Copilot Keys (Optional – system works without them)
GEMINI_API_KEY="your-gemini-api-key"
GROQ_API_KEY="your-groq-api-key"
```

Run

```
# Development server
npm run dev

# Production build
npm run build
npm run preview
```

Open <http://localhost:3000> in your browser.

13. Conclusion

NyayaSetu is not just another court management system — it is a **comprehensive, AI-assisted decision support platform** purpose-built for the Indian judiciary. It combines:

- **Deterministic scheduling** that is fully transparent and explainable
- **AI-powered assistance** that is always optional, never opaque
- **Human-in-the-loop governance** that keeps judicial officers in control
- **India-specific statutory compliance** built into every algorithm
- **Bilingual accessibility** that makes the system usable across Hindi-speaking regions
- **Public transparency** that gives citizens visibility into their own case status

Every line of code is written with one goal: **to help courts work faster, fairer, and more transparently — while never taking the human out of the loop.**

This report was generated for the NyayaSetu project, built for the Smart India Hackathon (SIH) 2026.

© 2026 Team NyayaSetu — Sujal Javeri